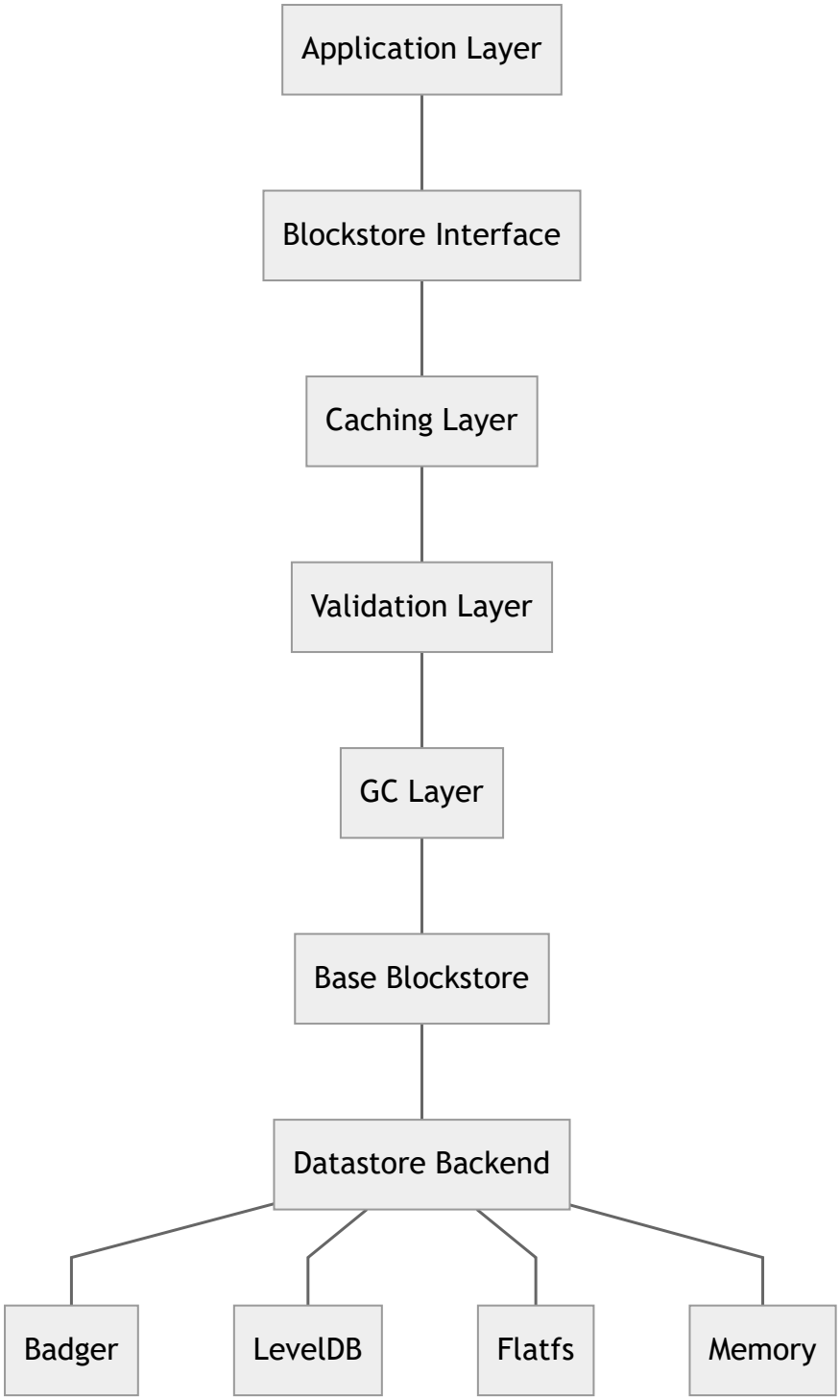
# 블록 저장소 아키텍처

## 개요

IPFS의 블록 저장소(Blockstore)는 콘텐츠 주소 지정 방식으로 데이터 블록을 저장하고 관리하는 핵심 컴포넌트입니다. 블록 저장소는 다양한 백엔드 구현을 지원하며, 각각 고유한 성능 특성과 사용 사례를 가지고 있습니다.

## 블록 저장소 계층 구조

IPFS의 블록 저장소는 여러 계층으로 구성되어 있으며, 각 계층은 특정한 기능을 담당합니다:




## 계층별 역할

1. **Application Layer**: 애플리케이션에서 블록 저장소에 접근하는 인터페이스
2. **Caching Layer**: 자주 사용되는 블록을 메모리에 캐시하여 성능 향상
3. **Validation Layer**: 블록의 무결성을 검증하고 해시 검증 수행
4. **GC Layer**: 가비지 컬렉션을 위한 락 메커니즘 제공

5. **Base Blockstore**: 기본 블록 저장소 구현
6. **Datastore Backend**: 실제 데이터를 저장하는 백엔드 구현

# 주요 블록 저장소 백엔드

## 1. Badger

**특징:**

- Go로 작성된 임베디드 키-값 데이터베이스
- LSM-tree 기반 구조로 높은 쓰기 성능
- 메모리 매핑을 통한 빠른 읽기 성능

**성능 특성:**

- 쓰기 성능: 매우 높음
- 읽기 성능: 높음
- 메모리 사용량: 중간
- 디스크 사용량: 효율적

**사용 사례:**

- 높은 쓰기 처리량이 필요한 환경
- 메모리가 충분한 시스템
- 일반적인 IPFS 노드 운영

## 2. LevelDB

**특징:**

- Google에서 개발한 키-값 저장소
- LSM-tree 기반으로 안정적인 성능

- 압축을 통한 효율적인 저장 공간 사용

**성능 특성:**

- 쓰기 성능: 높음
- 읽기 성능: 중간
- 메모리 사용량: 낮음
- 디스크 사용량: 압축으로 효율적

**사용 사례:**

- 안정성이 중요한 환경
- 메모리가 제한적인 시스템
- 장기간 운영되는 노드

## 3. Flatfs

**특징:**

- 파일 시스템 기반 저장소
- 각 블록을 개별 파일로 저장
- 단순한 구조로 디버깅이 용이

**성능 특성:**

- 쓰기 성능: 중간
- 읽기 성능: 파일 시스템에 의존
- 메모리 사용량: 낮음
- 디스크 사용량: 오버헤드 존재

**사용 사례:**

- 디버깅 및 개발 환경

- 파일 시스템 기반 백업이 필요한 경우
- 소규모 데이터 저장

### 4. Memory

**특징:**

- 메모리 기반 저장소
- 휘발성 데이터 저장
- 가장 빠른 접근 속도

**성능 특성:**

- 쓰기 성능: 매우 높음
- 읽기 성능: 매우 높음
- 메모리 사용량: 높음
- 지속성: 없음

**사용 사례:**

- 테스트 환경
- 임시 데이터 저장
- 캐시 용도

## 블록 저장소 구현 세부사항

### BaseBlockstore 생성

```
func  BaseBlockstoreCtor(
cacheOpts blockstore.CacheOpts,
```

```
hashOnRead  bool,

writeThrough  bool,

providingStrategy  string,

) func(mctx helpers.MetricsCtx, repo repo.Repo, prov provider.System, lc fx.Life

return  func(mctx helpers.MetricsCtx, repo repo.Repo, prov provider.System, lc f

opts := []blockstore.Option{blockstore.WriteThrough(writeThrough)}

// 제공 전략에 따른 블록 제공 설정

strategyFlag := config.ParseReproviderStrategy(providingStrategy)

shouldProvide := config.ReproviderStrategyAll | config.ReproviderStrategyFlat

if  strategyFlag&shouldProvide != 0 {

opts = append(opts, blockstore.Provider(prov))

}

// 기본 블록스토어 생성

bs = blockstore.NewBlockstore(repo.Datastore(), opts...)

// 검증 계층 추가

bs = &verifbs.VerifBS{Blockstore: bs}

// 캐싱 계층 추가

bs, err = blockstore.CachedBlockstore(helpers.LifecycleCtx(mctx, lc), bs, cacheO

if  err != nil {

return  nil, err

}

// ID 저장소 계층 추가

bs = blockstore.NewIdStore(bs)
```

```
// 읽기 시 해시 검증 옵션
if  hashOnRead {
bs = &blockstore.ValidatingBlockstore{Blockstore: bs}
}
return
}
}
```

## GC 지원 블록스토어

```
func  GcBlockstoreCtor(bb BaseBlocks) (gclocker blockstore.GCLocker, gcbs blocks
gclocker = blockstore.NewGCLocker()
gcbs = blockstore.NewGCBlockstore(bb, gclocker)
bs = gcbs
return
}
```

## 파일스토어 지원

```
func  FilestoreBlockstoreCtor(repo repo.Repo, bb BaseBlocks) (gclocker blockstor
gclocker = blockstore.NewGCLocker()
// 파일스토어 생성
fstore = filestore.NewFilestore(bb, repo.FileManager())
```

```
gcbs = blockstore.NewGCBlockstore(fstore, gclocker)

gcbs = &verifbs.VerifBSGC{GCBlockstore: gcbs}

bs = gcbs

return

}
```

## 가비지 컬렉션 메커니즘

IPFS의 가비지 컬렉션은 Mark-and-Sweep 알고리즘을 사용하여 사용하지 않는 블록을 정리합니다.

### GC 프로세스

1. **Mark 단계**: 보존해야 할 블록들을 마킹

- 재귀적으로 고정된(pinned) 블록들과 그 하위 블록들
- 직접 고정된 블록들
- 피너가 내부적으로 사용하는 블록들
- Best-effort 루트 블록들과 그 하위 블록들

2. **Sweep 단계**: 마킹되지 않은 블록들을 삭제

- 블록스토어의 모든 키를 순회
- 마킹되지 않은 블록들을 삭제
- 데이터스토어 레벨의 GC 실행

### GC 구현 세부사항

```
func GC(ctx context.Context, bs bstore.GCBlockstore, dstor dstore.Datastore, pn
// GC 락 획득
unlocker := bs.GCLock(ctx)
defer unlocker.Unlock(ctx)
// 보존할 블록 집합 생성
gcs, err := ColoredSet(ctx, pn, ds, bestEffortRoots, output)
if err != nil {
return err
}
// 모든 키 순회하며 삭제
keychain, err := bs.AllKeysChan(ctx)
for k := range keychain {
if !gcs.Has(k) {
err := bs.DeleteBlock(ctx, k)
// 결과 채널로 삭제 결과 전송
}
}
// 데이터스토어 레벨 GC
if gds, ok := dstor.(dstore.GCDatastore); ok {
err = gds.CollectGarbage(ctx)
}
```

```
}
```

# 저장 공간 관리

## 설정 옵션

```
type Datastore struct {
StorageMax  string  // 최대 저장 용량 (B, kB, MB 등)
StorageGCWatermark  int64  // GC 실행 임계점 (백분율)
GCPeriod  string  // GC 실행 주기
HashOnRead  bool  // 읽기 시 해시 검증
BloomFilterSize  int  // 블룸 필터 크기
BlockKeyCacheSize OptionalInteger // 블록 키 캐시 크기
WriteThrough Flag // 쓰기 관통 모드
}
```

## 캐싱 전략

1. **블록 키 캐시**: 블록 키와 크기를 캐시하여 존재 여부 확인 성능 향상
2. **Two-Queue 캐시**: LRU와 LFU를 결합한 캐시 알고리즘 사용
3. **쓰기 관통 모드**: 읽기 확인 없이 바로 쓰기하여 성능 향상

## 성능 최적화

1. **배치 처리**: 여러 블록을 한 번에 처리하여 I/O 오버헤드 감소

2. **비동기 처리**: 블록 제공과 저장을 비동기로 처리

3. **압축**: 데이터 압축을 통한 저장 공간 효율성 향상

4. **인덱싱**: 빠른 블록 검색을 위한 인덱스 구조 활용

# 모니터링 및 메트릭

## 주요 메트릭

- 블록 저장/검색 지연시간
- 캐시 히트율
- GC 실행 빈도 및 소요 시간
- 저장 공간 사용량
- 블록 제공 성능

## 로깅

블록 저장소는 다음과 같은 이벤트들을 로깅합니다:

- 블록 저장/삭제 작업
- GC 실행 결과
- 캐시 성능 통계
- 오류 및 경고 메시지

이러한 아키텍처를 통해 IPFS는 다양한 환경과 요구사항에 맞는 유연하고 효율적인 블록 저장소 시스템을 제공합니다.